

Department of Statistics & Computer Science
University of Kelaniya
Academic Year -2024/2025 (Semester II)

COSC 32133/BECS 32263

Full-Stack Software Development

Lab Sheet 06

Relational Database Design, Normalization (up to 3NF) and ORM with JPA

This lab is about databases. You will see why a single big table causes problems, normalize it up to Third Normal Form (3NF), create the database in MySQL, prove that a relational database has a fixed schema, and then use JPA (an ORM) in Spring Boot to map the tables to Java classes and read the data with custom query methods. There are no transactions and no web forms in this lab: the focus is the database.

1. Learning Outcomes

By the end of this lab, you will be able to:

- Explain what a relational database and its fixed schema are, and how this differs from schema-less (document) databases.
- Identify redundancy and the update, insert and delete anomalies in an unnormalized table.
- Normalize a table step by step to 1NF, 2NF and 3NF using functional dependencies.
- Create a MySQL database with tables, primary keys, foreign keys and constraints using SQL.
- Demonstrate that the schema is fixed: wrong columns, types, duplicates and broken references are rejected.
- Map tables to Java entities with JPA annotations, including @ManyToOne and @OneToMany relationships.
- Explain Hibernate's ddl-auto modes and use validate to check entities against the schema.
- Write custom repository query methods (derived queries and @Query) and relate each to SQL.

2. Scenario

A university keeps course registrations in one big spreadsheet-like table. Every row repeats the student's details, the course details, the lecturer's details and the department name. As more data is added, the table becomes inconsistent and hard to maintain. You have been asked to redesign it as a proper relational database and to build a small Spring Boot application that reads the data through JPA.

3. Prerequisites

- Lab Sheets 04 and 06 (Spring Boot, Spring Data JPA, MySQL connection, Lombok, GitHub workflow).
- **XAMPP** (Apache + MySQL) with **phpMyAdmin**, or MySQL Server with MySQL Workbench.
- **JDK 17 or later** (21 recommended), an IDE with Lombok support, Git and a GitHub account.

4. Key Concepts

4.1 Relational databases and the fixed schema

A relational database stores data in **tables** (rows and columns). Before any data is stored you must define the **schema**: the table names, the columns, the data type of every column, and the rules (keys and constraints). The database **enforces** the schema on every insert and update. This is called **schema-on-write**. Changing the structure requires an explicit ALTER TABLE command that affects every row.

	Relational (MySQL)	Document database (e.g. MongoDB)
Structure	Fixed schema defined in advance	Flexible: each document may have different fields
Add a new field	ALTER TABLE; every existing row gets NULL or a default	Just store a document with the new field
Data types	Enforced for every column	Not enforced by the database (by default)
Relationships	Foreign keys; the database guarantees valid references	Usually, embedded documents or manual references
Duplicate data	Avoided by normalization	Often accepted (denormalized) for speed
Best for	Structured data with strict rules: banking, registration, inventory	Rapidly changing or varied data: logs, catalogues, content

4.2 Keys, constraints and data types

Concept	Meaning
Primary key (PK)	Uniquely identify each row; never NULL. Example: student.id.
Foreign key (FK)	A column that must match a primary key in another table. Example: lecturer.department_id refers to department.id. It guarantees **referential integrity**.
UNIQUE	No two rows may have the same value, e.g. student.email.
NOT NULL	A value is required.
CHECK	A rule on the value, e.g. credits BETWEEN 1 AND 6.
Data type	INT, BIGINT, VARCHAR(n), DECIMAL, DATE ... The database rejects values of the wrong type or size.
Surrogate key	An artificial key such as an auto-incremented id. We use it in addition to natural identifiers such as student_code, which stay UNIQUE.

4.3 Why normalize?

Normalization organizes data into several related tables so that each fact is stored **once**. An abnormalized table suffers from:

- **Redundancy:** the same lecturer e-mail is stored in many rows.

- **Update anomaly:** changing a fact in one row but not in the others makes the data inconsistent.
- **Insert anomaly:** you cannot store a fact (a new course) until an unrelated fact exists (a student).
- **Delete anomaly:** deleting one fact (the last student of a course) accidentally destroys another (the course itself).

Form	Rule	How to fix a violation
1NF	Every cell holds **one atomic value**; no repeating groups; the table has a primary key.	Give every combination its own row; choose a (composite) primary key.
2NF	1NF **and** no **partial dependency**: every non-key column depends on the **whole** primary key (matters only for composite keys).	Move columns that depend on part of the key into their own table.
3NF	2NF **and** no **transitive dependency**: non-key columns depend on the key and **nothing but the key**.	Move columns that depend on another non-key column into their own table.

> **Memory aid:** Every non-key attribute must depend on **the key** (1NF), **the whole key** (2NF) and **nothing but the key** (3NF).

A **functional dependency** A → B means: if you know A you can determine exactly one B. For example StudentID → StudentName.

4.4 ORM and JPA

ORM (Object-Relational Mapping) connects the relational world (tables) with the object world (Java classes). **JPA** is the Java standard for ORM and **Hibernate** is the implementation used by Spring Boot. Spring Data JPA adds repositories on top.

Database	Java (JPA)	Annotation
Table	Class (entity)	@Entity, @Table(name = "student")
Row	Object (instance)	new Student()
Column	Field	@Column(name = "full_name", nullable = false)
Primary key	Field	@Id, @GeneratedValue(strategy = IDENTITY)
Foreign key, many-to-one	Reference to another entity	@ManyToOne, @JoinColumn(name = "department_id")
Other side, one-to-many	List of entities	@OneToMany(mappedBy = "department")

Hibernate can also create or change tables from your entities. This is controlled by spring.jpa.hibernate.ddl-auto:

Value	Behaviour
none	Hibernate does nothing to the schema.
validate	**Checks** that the tables and columns needed by the entities exist with compatible types. Never changes anything; the application refuses to start on a mismatch. **Used in this lab.**
update	Adds missing tables and columns automatically (never removes anything). Convenient in development but risky in production.
create	**Drops and recreates** all tables at start-up: all data is lost.
create-drop	Like create, and drops the tables again when the application stops.

> **Warning:** Never use create or create-drop on this lab's database. They would delete your tables and data.

5. The Unnormalized Data

This is what the registration office keeps today. Each student row contains **several courses inside one cell** (a repeating group). Assume: a course has one lecturer, a lecturer has one e-mail address and belongs to one department, and a department code identifies one department name.

Student ID	Student Name	Student Email	Course Code	Course Title	Cr.	Lecturer	Lecturer Email	Dept Code	Dept Name	Grade
S001	Kamal Silva	kamal@example.com	CS101 CS102	Intro to Programming Database Systems	3 3	Dr. Perera Dr. Fernando	perera@uni.lk fernando@uni.lk	D01 D01	Computer Science Computer Science	A B+
S002	Nimali Perera	nimali@example.com	CS101 ST201	Intro to Programming Probability Theory	3 3	Dr. Perera Dr. Jayasinghe	perera@uni.lk jayasinghe@uni.lk	D01 D02	Computer Science Statistics	B A-

Student ID	Student Name	Student Email	Course Code	Course Title	Cr.	Lecturer	Lecturer Email	Dept Code	Dept Name	Grade
S003	Saman Fernando	saman@example.com	CS102	Database Systems	3	Dr. Fernando	fernando@uni.lk	D01	Computer Science	C+
			ST201	Probability Theory	3	Dr. Jayasinghe	jayasinghe@uni.lk	D02	Statistics	B
S004	Dilini Jayawardena	dilini@example.com	CS201	Web Development	4	Dr. Perera	perera@uni.lk	D01	Computer Science	A

6. Tasks

Task 1: Start MySQL and Open an SQL Tool

1. Open the **XAMPP Control Panel** and click **Start** next to **MySQL** (and **Apache** if you will use phpMyAdmin).
2. Open **phpMyAdmin** (http://localhost/phpmyadmin) and select the **SQL** tab
3. Run this statement to confirm the server works:

```
SHOW DATABASES;
```

Task 2: Convert to 1NF and Experience the Problems

1. **Convert to 1NF (in Ms Word).** In the table in Section 5, which cells break the rule ‘one atomic value per cell’? Rewrite the table so that every student/course combination is on its own row. Which two columns together form the primary key?
2. Compare your answer with the script below. It creates the database university_db and your 1NF table registration_flat with 7 rows. Run the whole script in the SQL tab:

```
-- ============================================================
-- Anomaly demo: ONE flat table (1NF only). Used in Task 3.
-- ============================================================
CREATE DATABASE IF NOT EXISTS university_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE university_db;

CREATE TABLE registration_flat (
  student_id      VARCHAR(10),
  student_name    VARCHAR(100),
  student_email   VARCHAR(100),
  course_code     VARCHAR(10),
  course_title    VARCHAR(100),
  credits         INT,
```

```
  lecturer_name    VARCHAR(100),
  lecturer_email   VARCHAR(100),
  department_code  VARCHAR(10),
  department_name  VARCHAR(100),
  grade            VARCHAR(2),
  PRIMARY KEY (student_id, course_code)
) ENGINE=InnoDB;

INSERT INTO registration_flat VALUES
 ('S001','Kamal Silva','kamal@example.com','CS101','Intro to Programming',3,'Dr.
Perera','perera@uni.lk','D01','Computer Science','A'),
 ('S001','Kamal Silva','kamal@example.com','CS102','Database Systems',3,'Dr.
Fernando','fernando@uni.lk','D01','Computer Science','B+'),
 ('S002','Nimali Perera','nimali@example.com','CS101','Intro to Programming',3,'Dr.
Perera','perera@uni.lk','D01','Computer Science','B'),
 ('S002','Nimali Perera','nimali@example.com','ST201','Probability Theory',3,'Dr.
Jayasinghe','jayasinghe@uni.lk','D02','Statistics','A-'),
 ('S003','Saman Fernando','saman@example.com','CS102','Database Systems',3,'Dr.
Fernando','fernando@uni.lk','D01','Computer Science','C+'),
 ('S003','Saman Fernando','saman@example.com','ST201','Probability Theory',3,'Dr.
Jayasinghe','jayasinghe@uni.lk','D02','Statistics','B'),
 ('S004','Dilini Jayawardena','dilini@example.com','CS201','Web Development',4,'Dr.
Perera','perera@uni.lk','D01','Computer Science','A');
```

3. Check the data:

```
SELECT * FROM registration_flat;
```

3. Cause **the anomalies.** Run the statements below **one at a time** (A, then B, then C) and note what happens. Take a screenshot of each result.

```
-- ================================================================
-- Run AFTER anomaly-demo. SQL. One statement at a time (Task 2).
-- ================================================================

USE university_db;

-- A. REDUNDANCY + UPDATE anomaly -------------------------------
-- How many times is Dr. Perera's e-mail stored?

SELECT COUNT (*) AS copies FROM registration_flat WHERE lecturer_email =
'perera@uni.lk';

-- Change the e-mail in ONE row only (a careless UPDATE)

UPDATE registration_flat
SET lecturer_email = 'perera.new@uni.lk'
WHERE student_id = 'S001' AND course_code = 'CS101';
```

```
-- Dr. Perera now has two different e-mails!

SELECT DISTINCT lecturer_name, lecturer_email
FROM registration_flat WHERE lecturer_name = 'Dr. Perera';

-- B. INSERT anomaly ----------------------------------------
-- A new course that has no students yet cannot be stored

INSERT INTO registration_flat
  (course_code, course_title, credits, lecturer_name, lecturer_email, department_code,
department_name)
VALUES
  ('CS301', 'Software Engineering', 3, 'Dr. Fernando', 'fernando@uni.lk', 'D01', 'Computer
Science');

-- C. DELETE anomaly ----------------------------------------------
-- Dilini (S004) is the only student of CS201 and leaves the university
DELETE FROM registration_flat WHERE student_id = 'S004';

-- The course CS201 (title, credits, lecturer) has vanished too
SELECT * FROM registration_flat WHERE course_code = 'CS201';
```

Anomaly	What you will see	Why it is a problem
Redundancy	Dr. Perera's e-mail is stored in **3** rows.	Wasted space; every copy must be kept identical.
Update	After the single UPDATE, Dr. Perera has **two** different e-mails.	The database now contradicts itself.
Insert	Error: Field 'student_id' doesn't have a default value.	A course cannot exist without a student.
Delete	The query for CS201 returns **no rows**.	Deleting a student destroyed the course and lecturer data.

Task 3: Normalize to 2NF and 3NF (Paper Exercise)

Work in your notebook. Write your answers before looking at **Appendix B**.

1. **List the functional dependencies** of the 1NF table. Fill in the right-hand sides:

```
StudentID              -> ?
CourseCode             -> ?
(StudentID, CourseCode)  -> ?
LecturerEmail          -> ?
DeptCode               -> ?
```

2. **2NF:** Which columns depend on only **part** of the key (StudentID alone or CourseCode alone)? Split the table so that every column depends on the whole key. Write the resulting tables with their primary keys.

3. **3NF:** In the 2NF tables, find a non-key column that depends on another non-key column (a **transitive dependency**; hint: look at the course table and follow CourseCode → LecturerEmail → ...). Split again. Write the final tables with primary keys and **foreign keys**.
4. Check that your result has **five** tables. Draw an ER diagram with crow's-foot lines (one-to-many) between them.
5. Compare your tables and diagram with **Appendix B** and correct your work.

Task 4: Create the 3NF Database

1. Delete the flat table (we no longer need it):

```
USE university_db;
DROP TABLE registration_flat;
```

2. Run the **schema script**. Notice how every rule of the design is written into the schema: keys, NOT NULL, UNIQUE, CHECK and foreign keys. Tables are created in an order so that referenced tables exist first:

```
-- ============================================================
--  university_db : 3NF schema (Lab Sheet 07)
-- ============================================================
CREATE DATABASE IF NOT EXISTS university_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE university_db;

CREATE TABLE department (
  id    BIGINT AUTO_INCREMENT PRIMARY KEY,
  code  VARCHAR(10)  NOT NULL UNIQUE,
  name  VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE lecturer (
  id           BIGINT AUTO_INCREMENT PRIMARY KEY,
  name         VARCHAR(100) NOT NULL,
  email        VARCHAR(100) NOT NULL UNIQUE,
  department_id BIGINT NOT NULL,
  CONSTRAINT fk_lecturer_department
    FOREIGN KEY (department_id) REFERENCES department (id)
) ENGINE=InnoDB;

CREATE TABLE course (
  id         BIGINT AUTO_INCREMENT PRIMARY KEY,
  code       VARCHAR(10)  NOT NULL UNIQUE,
  title      VARCHAR(100) NOT NULL,
  credits    INT NOT NULL CHECK (credits BETWEEN 1 AND 6),
  lecturer_id BIGINT NOT NULL,
  CONSTRAINT fk_course_lecturer
    FOREIGN KEY (lecturer_id) REFERENCES lecturer (id)
) ENGINE=InnoDB;
```

```
CREATE TABLE student (
  id           BIGINT AUTO_INCREMENT PRIMARY KEY,
  student_code VARCHAR(10)  NOT NULL UNIQUE,
  full_name    VARCHAR(100) NOT NULL,
  email        VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE enrollment (
  id         BIGINT AUTO_INCREMENT PRIMARY KEY,
  student_id BIGINT NOT NULL,
  course_id  BIGINT NOT NULL,
  grade      VARCHAR(2),
  CONSTRAINT fk_enrollment_student FOREIGN KEY (student_id) REFERENCES student
(id),
  CONSTRAINT fk_enrollment_course  FOREIGN KEY (course_id)  REFERENCES course
(id),
  CONSTRAINT uq_enrollment UNIQUE (student_id, course_id)
) ENGINE=InnoDB;
```

3. Run the **data script** on the new, empty tables:

```
-- ============================================================
--  Sample data (run on freshly created, EMPTY tables)
-- ============================================================
USE university_db;

INSERT INTO department (code, name) VALUES
  ('D01', 'Computer Science'),
  ('D02', 'Statistics');

INSERT INTO lecturer (name, email, department_id) VALUES
  ('Dr. Perera',     'perera@uni.lk',     1),
  ('Dr. Fernando',   'fernando@uni.lk',   1),
  ('Dr. Jayasinghe', 'jayasinghe@uni.lk', 2);

INSERT INTO course (code, title, credits, lecturer_id) VALUES
  ('CS101', 'Intro to Programming', 3, 1),
  ('CS102', 'Database Systems',     3, 2),
  ('ST201', 'Probability Theory',   3, 3),
  ('CS201', 'Web Development',      4, 1);

INSERT INTO student (student_code, full_name, email) VALUES
  ('S001', 'Kamal Silva',        'kamal@example.com'),
  ('S002', 'Nimali Perera',      'nimali@example.com'),
  ('S003', 'Saman Fernando',     'saman@example.com'),
  ('S004', 'Dilini Jayawardena', 'dilini@example.com');
```

```
INSERT INTO enrollment (student_id, course_id, grade) VALUES
  (1, 1, 'A'),
  (1, 2, 'B+'),
  (2, 1, 'B'),
  (2, 3, 'A-'),
  (3, 2, 'C+'),
  (3, 3, 'B'),
  (4, 4, 'A');
```

4. Inspect the structure and the data:

```
SHOW TABLES;
DESCRIBE student;
DESCRIBE enrollment;
SELECT * FROM course;
```

5. **Prove no information was lost.** A JOIN reassembles the original flat table from the five tables. Run it and compare with Section 5:

```
-- Rebuilds the original flat table from the five normalized tables
USE university_db;

SELECT s.student_code, s.full_name, s.email AS student_email,
       c.code AS course_code, c.title, c.credits,
       l.name AS lecturer, l.email AS lecturer_email,
       d.code AS dept_code, d.name AS dept_name,
       e.grade
FROM enrollment e
JOIN student    s ON s.id = e.student_id
JOIN course     c ON c.id = e.course_id
JOIN lecturer   l ON l.id = c.lecturer_id
JOIN department d ON d.id = l.department_id
ORDER BY s.student_code, c.code;
```

6. **See the ER diagram.** In phpMyAdmin select university_db → **More** → **Designer** (in MySQL Workbench use **Database** → **Reverse Engineer**). Arrange the tables and take a screenshot. It should look like Figure 1.

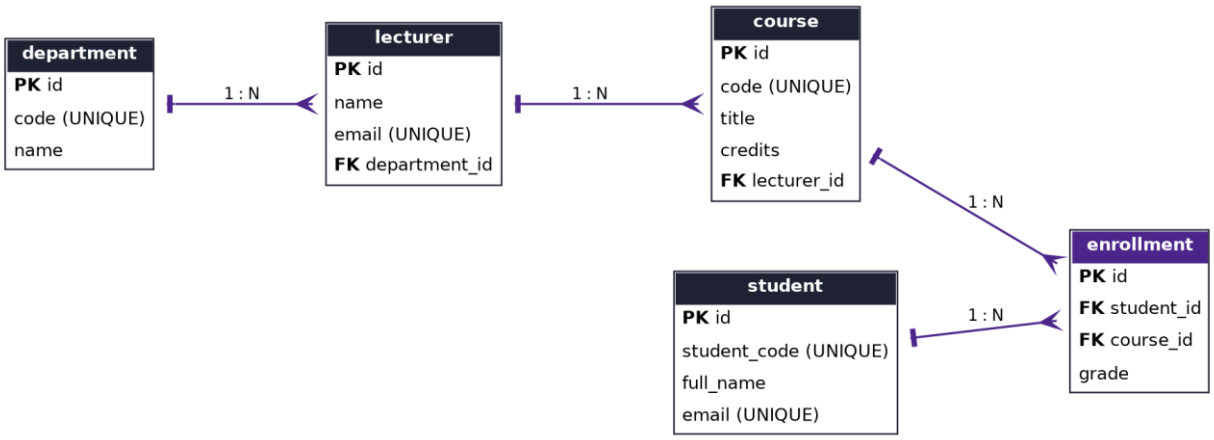


Figure 1: ER diagram of university_db (3NF)

Now repeat the three anomaly operations on the normalized design and notice that they are solved:

Anomaly	In the 3NF design
Update	Change Dr. Perera's e-mail in the **one** row of lecturer. Every course shows the new e-mail through the JOIN.
Insert	INSERT INTO course ... works without any student.
Delete	Deleting an enrollment removes only that enrollment. The course, lecturer and department stay.

Task 5: Prove that the Schema is Fixed

Open the file below and run each statement **one at a time**. **Every one of the first eight statements must fail.** Copy the error messages into your report.

```
-- ============================================================
-- Fixed-schema experiments (Task 5). Run ONE statement at a time
-- and read the error message. Every statement below should FAIL.
-- ============================================================
USE university_db;

-- 1. A column that is not in the schema
INSERT INTO student (student_code, full_name, email, phone)
VALUES ('S010', 'Test One', 'test1@example.com', '0771234567');

-- 2. A required (NOT NULL) column left out
INSERT INTO student (student_code, full_name) VALUES ('S011', 'Test Two');

-- 3. Wrong data type
INSERT INTO course (code, title, credits, lecturer_id) VALUES ('CS301', 'Test Course', 'three',
1);

-- 4. Duplicate value in a UNIQUE column
INSERT INTO student (student_code, full_name, email) VALUES ('S001', 'Test Three',
'test3@example.com');

-- 5. Value longer than the column allows
INSERT INTO department (code, name) VALUES ('D0000000001', 'Too Long Code');
```

```
-- 6. Foreign key pointing to a row that does not exist
INSERT INTO enrollment (student_id, course_id, grade) VALUES (1, 999, 'A');

-- 7. Deleting a row that other rows still refer to
DELETE FROM department WHERE code = 'D01';

-- 8. A value that breaks the CHECK constraint
INSERT INTO course (code, title, credits, lecturer_id) VALUES ('CS302', 'Test Course', 10, 1);

-- ---------------------------------------------------------------
--  Changing the structure must be done explicitly with ALTER TABLE
-- ---------------------------------------------------------------

ALTER TABLE student ADD COLUMN phone VARCHAR(15);
DESCRIBE student;
SELECT * FROM student;          -- existing rows get NULL in the new column


-- Undo the change (needed before Task 10)
ALTER TABLE student DROP COLUMN phone;
```

#	Statement	Typical MySQL error	Lesson
1	Insert into a column that does not exist	1054 Unknown column ‘phone’ in ‘field list’	Columns are fixed
2	Leave out a NOT NULL column	1364 Field ‘email’ doesn’t have a default value	Required values
3	Text in an INT column	1366 Incorrect integer value: ‘three’	Data types are enforced
4	Duplicate UNIQUE value	1062 Duplicate entry ‘S001’ for key ‘student_code’	Uniqueness
5	Text longer than VARCHAR(10)	1406 Data too long for column ‘code’	Size limits
6	Enrollment for a course that does not exist	1452 Cannot add or update a child row: a foreign key constraint fails	Referential integrity
7	Delete a department that has lecturers	1451 Cannot delete or update a parent row	Referential integrity
8	Credits = 10	3819 Check constraint ‘course_chk_1’ is violated	CHECK rules

> **XAMPP note:** XAMPP uses MariaDB, so some error numbers and wording differ slightly (for example 4025 for a CHECK violation). The meaning is the same.

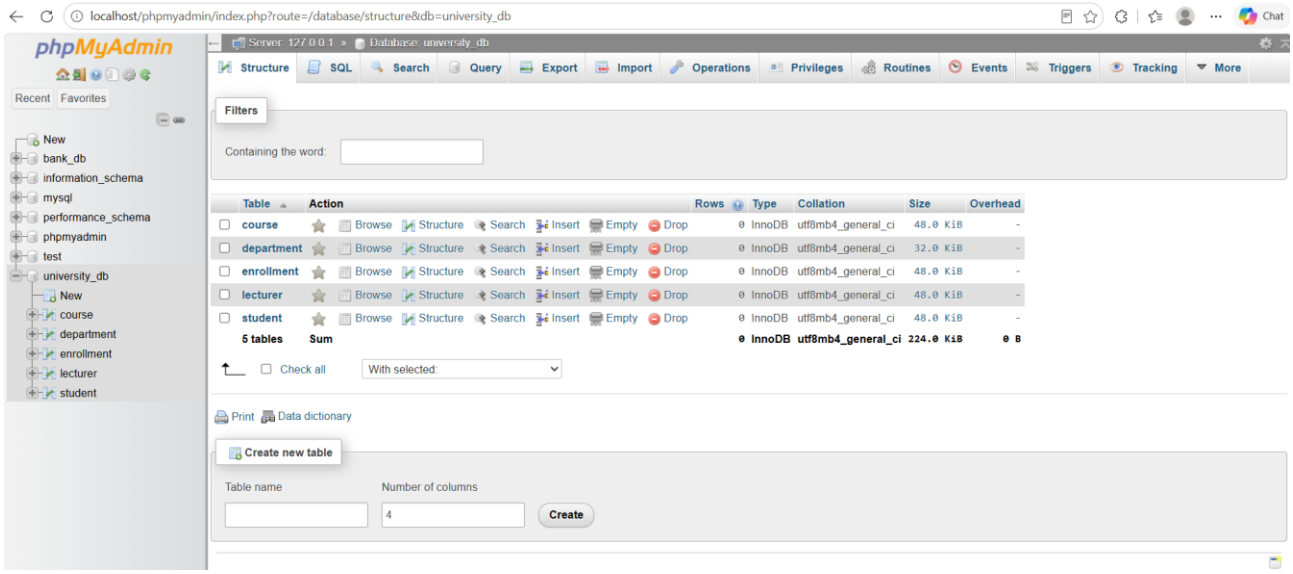
Finally, see how the structure is changed **deliberately**. The last lines of the script add a column and then remove it again:

```
ALTER TABLE student ADD COLUMN phone VARCHAR(15);
```

```
DESCRIBE student;
SELECT * FROM student;      -- every existing row now has phone = NULL

ALTER TABLE student DROP COLUMN phone;   -- undo (needed before Task 10)
```

Think: how would a document database such as MongoDB have handled the phone field in statement 1? What is gained and what is lost compared with a fixed schema?



Task 6: Create the Spring Boot Project and the GitHub Repository

1. Open **https://start.spring.io** with: Maven, Java, latest stable Spring Boot, Group com.example, Artifact university-db-app, Package name com.example.universitydb, Packaging Jar, Java 21 (or 17).

Dependency	Added to pom.xml	Why
Spring Web	spring-boot-starter-web (-webmvc in Boot 4)	REST endpoints to view the query results
Spring Data JPA	spring-boot-starter-data-jpa	JPA, Hibernate and repositories (the ORM)
MySQL Driver	mysql-connector-j	Connects Java to MySQL
Lombok	lombok	Generates getters and setters

2. Click **GENERATE**, extract the zip, open the folder in your IDE and enable Lombok annotation processing.
3. Create a folder sql in the project root and save schema.sql, data.sql, anomaly-demo.sql, anomaly-tests.sql, schema-violations.sql and join-check.sql in it. **Your database design is part of your project and must be in Git.**
4. Create a GitHub repository university-db-app (no README) and push the initial project:

```
git init
git add .
git commit -m "Initial project and SQL scripts"
git branch -M main
git remote add origin https://github.com/<your-username>/university-db-app.git
```

```
git push -u origin main
git switch -c feature/jpa-mapping
```

Task 7: Configure the Connection (ddl-auto = validate)

Replace the content of src/main/resources/application.properties. Notice that **you** created the database in Task 4; Spring Boot does not create it. With XAMPP the user is root and the password is empty. (If your MySQL has a password, put it in secrets.properties as in Lab Sheet 04 and add that file to .gitignore.)

```
spring.application.name=university-db-app

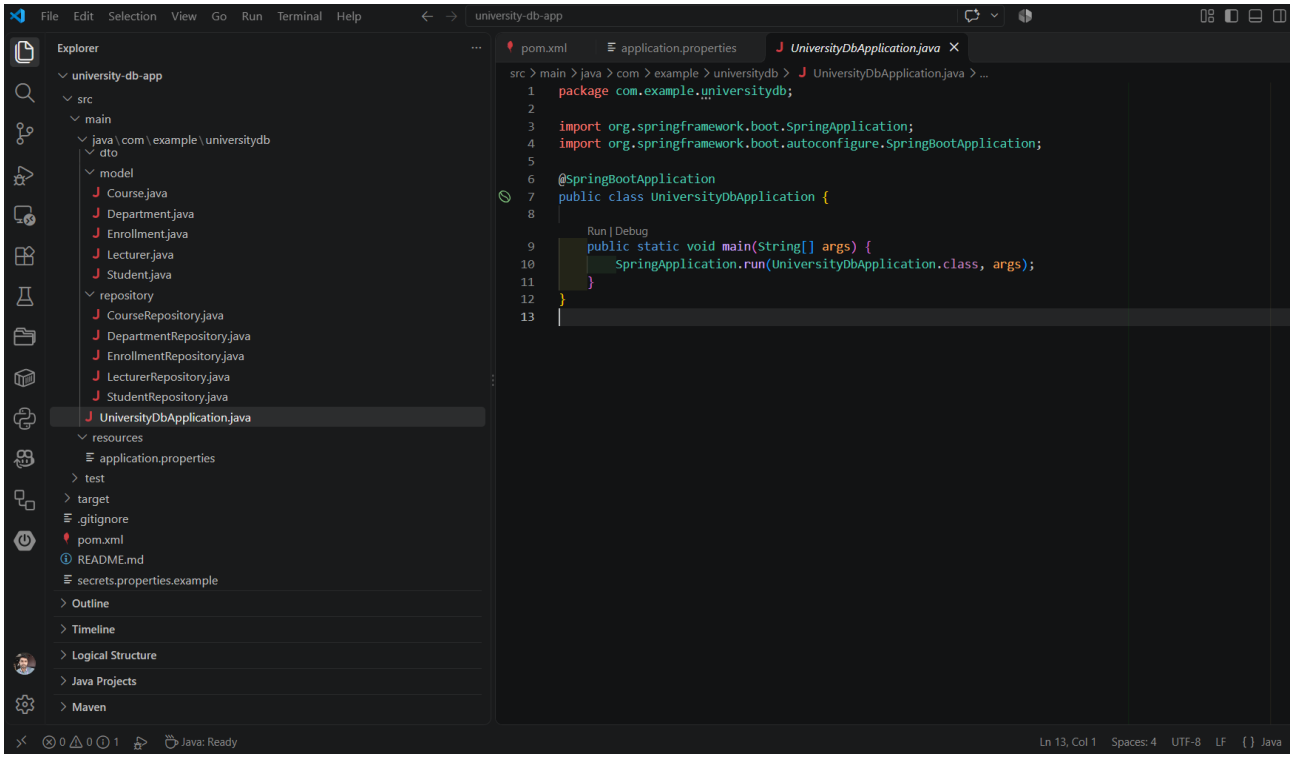
# ---- MySQL connection (XAMPP defaults: user root, empty password) ----
# The database "university_db" is created by YOU with sql/schema.sql (not by Spring Boot)
spring.datasource.url=jdbc:mysql://localhost:3306/university_db?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=

# ---- JPA / Hibernate ----
# validate = only CHECK that the entities match the existing tables. Never change the schema.
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.open-in-view=false
```

> **Schema first:** In this lab the **database** is the master and the Java classes must match it. With validate, Hibernate will never create or change tables. If an entity does not fit the schema, the application stops with an error.

Task 8: Map the Tables to Entities

Create the package model and the five entity classes. Every field uses an explicit @Column(name = ...) so that you can see how the column in the table corresponds to the field in the class. Study the relationship annotations in the comments.



8.1 Department.java

```
package com.example.universitydb.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
import jakarta.persistence.Table;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "department")
@Getter
@Setter
@NoArgsConstructor
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;

    @Column(name = "code", nullable = false, unique = true, length = 10)
    private String code;

    @Column(name = "name", nullable = false, length = 100)
    private String name;

    // ONE department has MANY lecturers. "mappedBy" = the foreign key lives in the lecturer
table.
    @OneToMany(mappedBy = "department")
    private List<Lecturer> lecturers = new ArrayList<>();
}
```

8.2 Lecturer.java

```
package com.example.universitydb.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Entity
@Table(name = "lecturer")
@Getter
@Setter
@NoArgsConstructor
public class Lecturer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "name", nullable = false, length = 100)
    private String name;

    @Column(name = "email", nullable = false, unique = true, length = 100)
    private String email;

    // MANY lecturers belong to ONE department (foreign key column: department_id)
    @ManyToOne(optional = false)
    @JoinColumn(name = "department_id")
    private Department department;
```

```
}
```

8.3 Course.java

```
package com.example.universitydb.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Entity
@Table(name = "course")
@Getter
@Setter
@NoArgsConstructor
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "code", nullable = false, unique = true, length = 10)
    private String code;

    @Column(name = "title", nullable = false, length = 100)
    private String title;

    @Column(name = "credits", nullable = false)
    private Integer credits;

    // MANY courses are taught by ONE lecturer (foreign key column: lecturer_id)
    @ManyToOne(optional = false)
    @JoinColumn(name = "lecturer_id")
    private Lecturer lecturer;
}
```

8.4 Student.java

```
package com.example.universitydb.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
```

```
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Entity
@Table(name = "student")
@Getter
@Setter
@NoArgsConstructor
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "student_code", nullable = false, unique = true, length = 10)
    private String studentCode;

    @Column(name = "full_name", nullable = false, length = 100)
    private String fullName;

    @Column(name = "email", nullable = false, unique = true, length = 100)
    private String email;
}
```

8.5 Enrollment.java

```
package com.example.universitydb.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import jakarta.persistence.UniqueConstraint;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

/** Link table between student and course (a many-to-many relationship with its own column:
grade). */
@Entity
@Table(name = "enrollment",
      uniqueConstraints = @UniqueConstraint(name = "uq_enrollment", columnNames =
{"student_id", "course_id"}))
```

```
@Getter
@Setter
@NoArgsConstructor
public class Enrollment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false)
    @JoinColumn(name = "student_id")
    private Student student;

    @ManyToOne(optional = false)
    @JoinColumn(name = "course_id")
    private Course course;

    @Column(name = "grade", length = 2)
    private String grade;
}
```

Relationship in the database	Mapping in Java
lecturer.department_id → department.id	Lecturer: @ManyToOne + @JoinColumn(name = "department_id"). Department: @OneToMany(mappedBy = "department").
course.lecturer_id → lecturer.id	Course: @ManyToOne + @JoinColumn(name = "lecturer_id")
enrollment.student_id, enrollment.course_id	Enrollment: two @ManyToOne references. The link table gets its own entity because it has its own column (grade).

```
git add .
git commit -m "Map database tables to JPA entities"
```

Task 9: Run the Application and Test the Schema Check

1. With MySQL running, start the application:
2. It must start without errors: Hibernate has **validated** that all five entities match the tables.
3. **Experiment A (fixed schema):** stop the application. In Student.java add a new field (and the import jakarta.persistence.Column is already there):

```
@Column(name = "phone", length = 15)
private String phone;
```

4. Start the application again. It now **fails to start** with an error similar to the one below. The Java class asked for a column that the fixed schema does not have. Take a screenshot.

```
Schema-validation: missing column [phone] in table [student]
```

5. **Experiment B (changing the schema):** set spring.jpa.hibernate.ddl-auto=update, start the application and check the database. Hibernate has executed an ALTER TABLE for you (look at the SQL in the console and run DESCRIBE student;). Note that the structure changed for the **whole table**, and all existing students have phone = NULL.
6. **Restore the lab state:** stop the application, remove the phone field from Student.java, set ddl-auto back to validate, and drop the column in MySQL:

```
ALTER TABLE student DROP COLUMN phone;
```

7. Start the application once more to confirm that it runs again.

Task 10: Write the Repositories with Custom Query Methods

A repository is an interface. Spring Data JPA creates the implementation and **translates the method name into SQL**. The rule is: find + By + property name + keyword. Create the package repository and the five interfaces (the SQL in the comments is what Hibernate generates):

10.1 StudentRepository.java

```
package com.example.universitydb.repository;

import com.example.universitydb.model.Student;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.Optional;

public interface StudentRepository extends JpaRepository<Student, Long> {

    // SELECT ... FROM student WHERE student_code = ?
    Optional<Student> findByStudentCode(String studentCode);

    // WHERE LOWER(full_name) LIKE LOWER('%?%')
    List<Student> findByFullNameContainingIgnoreCase(String text);

    // ORDER BY full_name ASC
    List<Student> findAllByOrderByFullNameAsc();

    // WHERE email = ?  (returns true / false)
    boolean existsByEmail(String email);

    // WHERE email LIKE '%?'  (count rows)
    long countByEmailEndingWith(String domain);
}
```

10.2 CourseRepository.java

```
package com.example.universitydb.repository;

import com.example.universitydb.model.Course;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
```

```
import java.util.Optional;

public interface CourseRepository extends JpaRepository<Course, Long> {

    Optional<Course> findByCode(String code);

    // WHERE credits >= ?
    List<Course> findByCreditsGreaterThanEqual(Integer credits);

    List<Course> findByTitleContainingIgnoreCase(String text);

    // two relationships deep: Course -> lecturer -> name   (SQL JOIN is generated for you)
    List<Course> findByLecturer_Name(String lecturerName);

    // three levels: Course -> lecturer -> department -> name  (two JOINs)
    List<Course> findByLecturer_Department_Name(String departmentName);
}
```

10.3 LecturerRepository.java

```
package com.example.universitydb.repository;

import com.example.universitydb.model.Lecturer;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.Optional;

public interface LecturerRepository extends JpaRepository<Lecturer, Long> {

    Optional<Lecturer> findByEmail(String email);

    // property path: Lecturer.department.code  (underscore = go into the related object)
    List<Lecturer> findByDepartment_Code(String departmentCode);
}
```

10.4 DepartmentRepository.java

```
package com.example.universitydb.repository;

import com.example.universitydb.model.Department;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.Optional;

public interface DepartmentRepository extends JpaRepository<Department, Long> {

    Optional<Department> findByCode(String code);
```

```
    // JOIN FETCH loads the department AND its lecturers with ONE SQL query
    @Query("SELECT d FROM Department d LEFT JOIN FETCH d.lecturers WHERE d.code =
:code")
    Optional<Department> findWithLecturers(@Param("code") String code);
}
```

10.5 EnrollmentRepository.java

```
package com.example.universitydb.repository;

import com.example.universitydb.dto.CourseEnrollmentCount;
import com.example.universitydb.model.Enrollment;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

import java.util.Collection;
import java.util.List;

public interface EnrollmentRepository extends JpaRepository<Enrollment, Long> {

    List<Enrollment> findByStudent_StudentCode(String studentCode);

    List<Enrollment> findByCourse_Code(String courseCode);

    List<Enrollment> findByGrade(String grade);

    // WHERE grade IN (?, ?, ...)
    List<Enrollment> findByGradeIn(Collection<String> grades);

    // WHERE student.student_code = ? AND grade = ?
    List<Enrollment> findByStudent_StudentCodeAndGrade(String studentCode, String grade);

    long countByCourse_Code(String courseCode);

    // A report with GROUP BY: number of students in every course
    @Query("SELECT c.code AS courseCode, c.title AS title, COUNT(e) AS studentCount " +
           "FROM Enrollment e JOIN e.course c " +
           "GROUP BY c.code, c.title " +
           "ORDER BY COUNT(e) DESC, c.code")
    List<CourseEnrollmentCount> countStudentsPerCourse();
}
```

Query method keywords used

Method name part	Meaning	SQL produced
findByStudentCode	Exact match on a field	WHERE student_code = ?
...ContainingIgnoreCase	Text contains, ignoring case	WHERE LOWER(col) LIKE LOWER('%?%')
...GreaterThanEqual	Comparison	WHERE credits >= ?

Method name part	Meaning	SQL produced
...In	Value in a list	WHERE grade IN (?, ?)
...And...	Combine conditions	WHERE a = ? AND b = ?
...OrderByFullNameAsc	Sorting	ORDER BY full_name ASC
existsBy..., countBy...	Boolean / number result	SELECT COUNT(*) ...
findByLecturer_Department_Name	Property path through relationships (_)	JOIN lecturer ... JOIN department ... WHERE d.name = ?
@Query (JPQL)	Your own query when the name would be too long or you need GROUP BY / JOIN FETCH	Written by you, using **class and field names**

> **JPQL vs SQL:** JPQL (used in @Query) refers to entity classes and fields (Course c, c.code), not to tables and columns. JOIN FETCH tells Hibernate to load a department **and** its lecturers with one SQL query.

```
git add .
git commit -m "Add repositories with custom query methods"
```

Task 11: DTOs and a Read-only Controller

Entities are returned to the browser through small **DTO** records, so that only the needed fields are shown. Create the package dto with the six files below, and the package controller with UniversityController.

```
package com.example.universitydb.dto;

import com.example.universitydb.model.Student;

public record StudentDto(String studentCode, String fullName, String email) {

    public static StudentDto from(Student s) {
        return new StudentDto(s.getStudentCode(), s.getFullName(), s.getEmail());
    }
}
```

```
package com.example.universitydb.dto;

import com.example.universitydb.model.Lecturer;

public record LecturerDto(String name, String email, String department) {

    public static LecturerDto from(Lecturer l) {
        return new LecturerDto(l.getName(), l.getEmail(), l.getDepartment().getName());
    }
}
```

```
package com.example.universitydb.dto;

import com.example.universitydb.model.Course;

public record CourseDto(String code, String title, Integer credits, String lecturer, String
department) {

    public static CourseDto from(Course c) {
        return new CourseDto(c.getCode(), c.getTitle(), c.getCredits(),
                c.getLecturer().getName(), c.getLecturer().getDepartment().getName());
    }
}
```

```
package com.example.universitydb.dto;

import com.example.universitydb.model.Enrollment;

public record EnrollmentDto(String studentCode, String studentName,
                            String courseCode, String courseTitle, String grade) {

    public static EnrollmentDto from(Enrollment e) {
        return new EnrollmentDto(e.getStudent().getStudentCode(), e.getStudent().getFullName(),
                e.getCourse().getCode(), e.getCourse().getTitle(), e.getGrade());
    }
}
```

```
package com.example.universitydb.dto;

/** Interface projection: Spring fills these getters from the aliases in the @Query. */
public interface CourseEnrollmentCount {

    String getCourseCode();

    String getTitle();

    Long getStudentCount();
}
```

```
package com.example.universitydb.dto;

public record CourseCountDto(String courseCode, String title, Long studentCount) {

    public static CourseCountDto from(CourseEnrollmentCount p) {
        return new CourseCountDto(p.getCourseCode(), p.getTitle(), p.getStudentCount());
    }
}
```

UniversityController.java

```
package com.example.universitydb.controller;

import com.example.universitydb.dto.CourseCountDto;
import com.example.universitydb.dto.CourseDto;
import com.example.universitydb.dto.EnrollmentDto;
import com.example.universitydb.dto.LecturerDto;
import com.example.universitydb.dto.StudentDto;
import com.example.universitydb.model.Course;
import com.example.universitydb.repository.CourseRepository;
import com.example.universitydb.repository.DepartmentRepository;
import com.example.universitydb.repository.EnrollmentRepository;
import com.example.universitydb.repository.StudentRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

/** Read-only JSON endpoints that let you see the result of every repository query in the
browser. */
@RestController
@RequestMapping("/api")
@RequiredArgsConstructor
public class UniversityController {

    private final StudentRepository studentRepository;
    private final CourseRepository courseRepository;
    private final DepartmentRepository departmentRepository;
    private final EnrollmentRepository enrollmentRepository;

    // ---------- students ----------

    @GetMapping("/students")
    public List<StudentDto> students(@RequestParam(required = false) String name) {
        var list = (name == null || name.isBlank())
                ? studentRepository.findAllByOrderByFullNameAsc()
                : studentRepository.findByFullNameContainingIgnoreCase(name.trim());
        return list.stream().map(StudentDto::from).toList();
    }

    @GetMapping("/students/{code}")
    public ResponseEntity<StudentDto> student(@PathVariable String code) {
        return studentRepository.findByStudentCode(code)
                .map(s -> ResponseEntity.ok(StudentDto.from(s)))
                .orElse(ResponseEntity.notFound().build());
    }
```

```
    @GetMapping("/students/{code}/enrollments")
    public List<EnrollmentDto> studentEnrollments(@PathVariable String code) {
        return enrollmentRepository.findByStudent_StudentCode(code)
                .stream().map(EnrollmentDto::from).toList();
    }

    // ---------- courses ----------

    @GetMapping("/courses")
    public List<CourseDto> courses(@RequestParam(required = false) Integer minCredits,
                                   @RequestParam(required = false) String department) {
        List<Course> list;
        if (minCredits != null) {
            list = courseRepository.findByCreditsGreaterThanEqual(minCredits);
        } else if (department != null && !department.isBlank()) {
            list = courseRepository.findByLecturer_Department_Name(department.trim());
        } else {
            list = courseRepository.findAll();
        }
        return list.stream().map(CourseDto::from).toList();
    }

    @GetMapping("/courses/{code}/students")
    public List<StudentDto> courseStudents(@PathVariable String code) {
        return enrollmentRepository.findByCourse_Code(code)
                .stream().map(e -> StudentDto.from(e.getStudent())).toList();
    }

    // ---------- departments ----------

    @GetMapping("/departments/{code}/lecturers")
    public ResponseEntity<List<LecturerDto>> departmentLecturers(@PathVariable String
code) {
        return departmentRepository.findWithLecturers(code)
                .map(d ->
ResponseEntity.ok(d.getLecturers().stream().map(LecturerDto::from).toList()))
                .orElse(ResponseEntity.notFound().build());
    }

    // ---------- enrollments and reports ----------

    @GetMapping("/enrollments")
    public List<EnrollmentDto> enrollments(@RequestParam String grade) {
        return enrollmentRepository.findByGrade(grade)
                .stream().map(EnrollmentDto::from).toList();
    }

    @GetMapping("/reports/course-counts")
    public List<CourseCountDto> courseCounts() {
        return enrollmentRepository.countStudentsPerCourse()
                .stream().map(CourseCountDto::from).toList();
```

```
    }
}
```

```
git add .
git commit -m "Add DTOs and read-only REST controller"
```

Task 12: Run the Queries and Study the SQL

Start the application, open each URL in the browser (or Postman) and compare with the expected result. Keep an eye on the console: show-sql=true prints the SQL for every call.

URL (http://localhost:8080 ...)	Query method used	Expected result
/api/students	findAllByOrderByFullNameAsc	4 students, sorted by name (Dilini first)
/api/students?name=sil	...ContainingIgnoreCase	Kamal Silva only
/api/students/S001	findByStudentCode	Kamal Silva (404 for an unknown code)
/api/students/S001/enrollments	findByStudent_StudentCode	CS101 (A) and CS102 (B+)
/api/courses	findAll	4 courses with lecturer and department
/api/courses?minCredits=4	...GreaterThanEqual	CS201 only
/api/courses?department=Computer Science	findByLecturer_Department_Name	CS101, CS102, CS201
/api/courses/CS101/students	findByCourse_Code	Kamal Silva, Nimali Perera
/api/departments/D01/lecturers	findWithLecturers (JOIN FETCH)	Dr. Perera, Dr. Fernando
/api/enrollments?grade=A	findByGrade	S001/CS101 and S004/CS201
/api/reports/course-counts	countStudentsPerCourse (GROUP BY)	CS101: 2, CS102: 2, ST201: 2, CS201: 1

Example response of /api/courses?minCredits=4:

```
[
  {
    "code": "CS201",
    "title": "Web Development",
    "credits": 4,
    "lecturer": "Dr. Perera",
```

```
    "department": "Computer Science"
  }
]
```

Questions for your report (look at the console):

- For /api/courses?department=Computer Science, find the SQL statement. Which tables are joined?
- For /api/enrollments?grade=A, how many select statements ran? Why are there more than one? (@ManyToOne is loaded **eagerly** by default, so Hibernate fetches the student, course, lecturer and department of every row.)
- For /api/departments/D01/lecturers, how many statements ran? Why is this better? (JOIN FETCH.)

Task 13: Push to GitHub and Merge

1. Push your branch and create a Pull Request:

```
git status
git push -u origin feature/jpa-mapping
```

2. On GitHub create a **Pull Request** (base main, compare feature/jpa-mapping) with the title 'Add 3NF database design and JPA mapping'. Check that the **`sql/` folder** and all Java files are included and that target/ is not.
3. Merge the Pull Request, then update your local main:

```
git switch main
git pull origin main
git log --oneline --graph
```

Submission Requirements

- Submit as a PDF including all the screenshots and code. Include your name, student ID, date, course code, course name and your **GitHub repository URL**.
- Work from Task 3: functional dependencies, 2NF tables, 3NF tables and your ER diagram.
- Deadline: 2nd of November 2026